FACULDADE CAPITAL FEDERAL – UNIFECAF

Claudio Guilherme Gomes Alves

R.A 75955

Data Structure Strategy and Implementation

Resolução de problemas para os professores de administração.

TABOÃO DA SERRA/SP

2026

FACULDADE CAPITAL FEDERAL – UNIFECAF

Claudio Guilherme Gomes Alves
R.A. 75955

Data Structure Strategy and Implementation

Resolução de exercícios

Atividade apresentada ao curso de gestão da tecnologia da informação da faculdade UNIFECAF como requisito parcial para obtenção de conclusão do módulo de Data Structure Strategy and Implementation.

Prof. Vitor Jansen.

TABOÃO DA SERRA
2026

SUMÁRIO

1. Introdução.....3
2. Entendimento do desafio 3
3. Pesquisa e referência de mercado 4
4. Solução proposta 4
5. Implementação em Rust (visão geral) 4
6. Testes e validação 4
7. Desempenho e escalabilidade 4
8. Considerações finais 5
9. Referências 5

1. Introdução

Este trabalho apresenta uma implementação educacional de um sistema de busca otimizado para um catálogo de produtos, simulando o cenário de grande escala de um e-commerce. O objetivo foi reduzir o tempo de resposta e aumentar a relevância dos resultados, evitando varredura completa do catálogo a cada consulta. Adicionalmente, foi implementada recomendação baseada em grafo de similaridade.

2. Entendimento do desafio

A MegaStore possui um catálogo grande e um sistema de busca com baixa precisão e alta latência. A estratégia adotada foi: pré-processar e indexar o catálogo; realizar busca por termos via índice invertido; ordenar por relevância; aplicar filtros por marca/categoria; e gerar recomendações por similaridade.

3. Pesquisa e referência de mercado

Como referência prática, foram consultadas soluções utilizadas no mercado. Elasticsearch/Lucene utiliza índice invertido e modelos de ranking como BM25. OpenSearch é alternativa open-source/gerenciada que aplica princípios semelhantes. Aqui, os fundamentos são reproduzidos em versão enxuta, com fins didáticos.

4. Solução proposta

A solução foi dividida em busca e recomendação. Na busca, utiliza-se um índice invertido (termo → lista de produtos) e um ranking de relevância (BM25 simplificado). Na recomendação, modela-se um grafo em que cada produto é um nó e arestas representam similaridade por conteúdo (tokens).

5. Implementação em Rust (visão geral)

A implementação foi feita em Rust (edition 2021), priorizando segurança de memória e organização em módulos (models, text, index, search, graph, engine e CLI).

6. Testes e validação

Foram criados testes automatizados (integração/funcionais) na pasta tests/ para validar a CLI e o ranking de busca. Na execução local, o comando cargo test aprovou 3 testes (2 de CLI e 1 de busca)

Comandos utilizados:

cargo test
cargo run --release -- search --data data/sample_products.jsonl --q "iphone 128gb" --top-k 5
cargo run --release -- recommend --data data/sample_products.jsonl--id 1 --top-k 5

7. Desempenho e escalabilidade

Foi utilizado um benchmark sintético (bench) para medir tempos de indexação, construção do grafo e busca, variando N. Os valores abaixo foram obtidos em Windows, modo release.

N	Indexação	Grafo	Busca
5.000	30.8898 ms	664.1946 ms	52.1 µs
10.000	48.339 ms	9.4451769 s	157.9 µs

Interpretação: busca e indexação permanecem rápidas; a construção do grafo cresce mais rapidamente e tende a ser o gargalo. Em produção, recomenda-se construção incremental e/ou limitação de candidatos.

8. Considerações finais

O projeto demonstrou a aplicação de índice invertido e ranking de relevância em Rust, além de recomendação por grafo. Os resultados locais indicam baixa latência na busca e destacam o custo de construção do grafo como principal ponto de atenção para escalabilidade.

9. Referências

ELASTIC. Practical BM25.
https://www.elastic.co/blog/practical-bm25-part-2-the-bm25-algorithm-and-its-variables.

Acesso em: 04 mar. 2026.

OPENSEARCH. Documentation.
https://docs.opensearch.org/latest/.

Acesso em: 04 mar. 2026.

AWS. Amazon OpenSearch Service Documentation.
https://docs.aws.amazon.com/opensearch-service/.

Acesso em: 04 mar. 2026